



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Alan Rogelio Barrientos Ramírez

N° de Cuenta: 422022019

GRUPO DE LABORATORIO: 3

GRUPO DE TEORÍA: 6

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 12/09/2026

CALIFICACIÓN: _____

Cohete espacial y rombo de ocho pirámides

1 Ejecución de los ejercicios

Partí de practica3.cpp y de los shaders proporcionados en clase. Los dos ejercicios están en un solo main: con la tecla 1 se muestra el cohete y con la 2 el rombo. La tecla 3 permite revisar una pirámide por separado. Conservé las teclas E, R y T para girar los objetos y la proyección perspectiva de 60° con la relación de aspecto del ejemplo.

1.1 Cohete espacial

Para el cohete reutilicé las mallas del cilindro, cono, esfera, cubo y pirámide cuadrangular. El cuerpo central y los propulsores salen de cilindros; los conos forman las puntas, la tobera y las llamas. Las esferas forman la ventanilla, los cubos sujetan los propulsores y las pirámides funcionan como aletas.

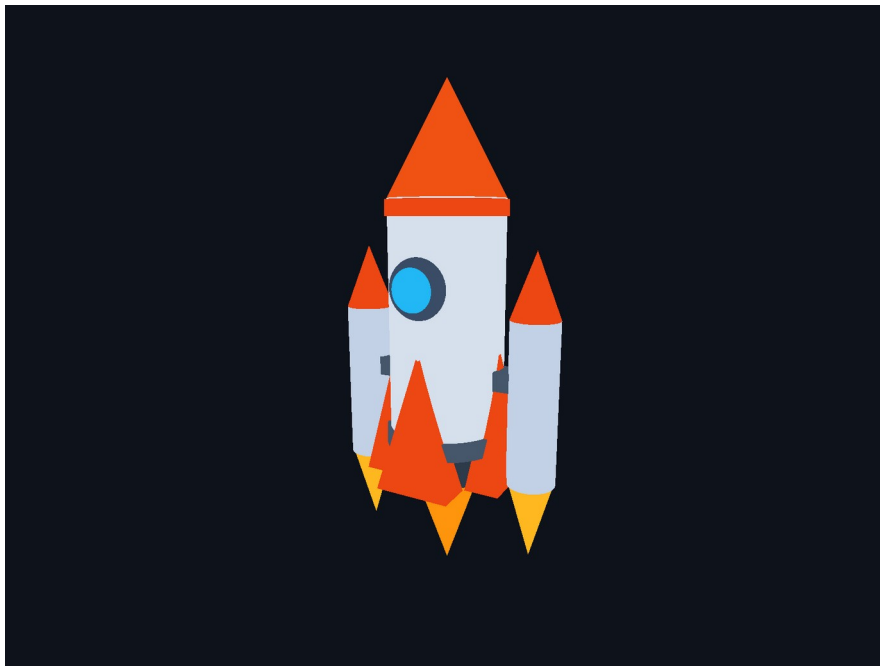


Figura 1. Ejecución del cohete construido con las cinco figuras solicitadas.

Malla	Instancias	Uso
Cilindro	5	Fuselaje, bandas y propulsores
Cono	7	Puntas, tobera y llamas
Esfera	2	Marco y cristal de la ventanilla
Cubo	2	Soportes laterales
Pirámide cuadrangular	4	Aletas

1.2 Bloques de dibujo y colores

Cada pieza empieza con la matriz identidad. Después apliqué las transformaciones del conjunto y las de la pieza, envié la matriz model y el color al shader, y dibujé la malla correspondiente. Repetí este bloque para las 20 instancias del cohete. La proyección no se cambió para acomodarlo: ajusté la escala y la posición de sus piezas.

```
practica3.cpp
428 // CILINDRO 01 - fuselaje central
429 model = glm::mat4(1.0);
430 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.80000f));
431 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
432 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
433 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
434 model = glm::translate(model, glm::vec3(0.00000f, 0.15000f, 0.00000f));
435 model = glm::scale(model, glm::vec3(0.48000f, 1.95000f, 0.48000f));
436 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
437 color = glm::vec3(0.84000f, 0.88000f, 0.93000f);
438 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
439 meshList[2]->RenderMesh();
```

Figura 2. Bloque del main que coloca y dibuja el cilindro central.

La pirámide de colores conserva las cinco posiciones y los seis triángulos de la pirámide cuadrangular del profesor. Repetí los vértices por cara para que cada triángulo tenga un RGB constante. Así, cada cara conserva un color uniforme, en lugar de interpolar colores distintos entre sus vértices. Utilicé MeshColor, que ya estaba incluida en Mesh.h.

```
practica3.cpp
139 // P01: las mismas posiciones e índices se expanden por cara.
140 // Cada vertice de una cara recibe el mismo RGB para evitar degradados.
141 // Orden: frontal roja, izquierda verde, trasera amarilla,
142 // derecha magenta y los dos triangulos de la base azul.
143 GLfloat verticesPiramideColor[] = {
144     0.50000f, -0.50000f, 0.50000f, 1.00000f, 0.00000f, 0.00000f,
145     -0.50000f, -0.50000f, 0.50000f, 1.00000f, 0.00000f, 0.00000f,
146     0.00000f, 0.50000f, 0.00000f, 1.00000f, 0.00000f, 0.00000f,
```

Figura 3. Posiciones originales repetidas por cara y color rojo de la primera cara.

Cara de la malla original	Color	RGB
Frontal	Rojo	(1, 0, 0)
Izquierda	Verde	(0, 1, 0)
Trasera	Amarillo	(1, 1, 0)
Derecha	Magenta	(1, 0, 1)
Base cuadrada	Azul	(0, 0, 1)

1.3 Unión de las ocho pirámides

El segundo ejercicio tiene ocho instancias de la pirámide de colores: cuatro arriba y cuatro abajo. Las superiores comparten la punta $(0, 1, 0)$, y las inferiores la punta $(0, -1, 0)$. Sus bases cuadradas cubren el plano central $Y = 0$. El resultado es un rombo tridimensional cerrado, sin las puntas separadas que aparecían en los primeros acomodos.

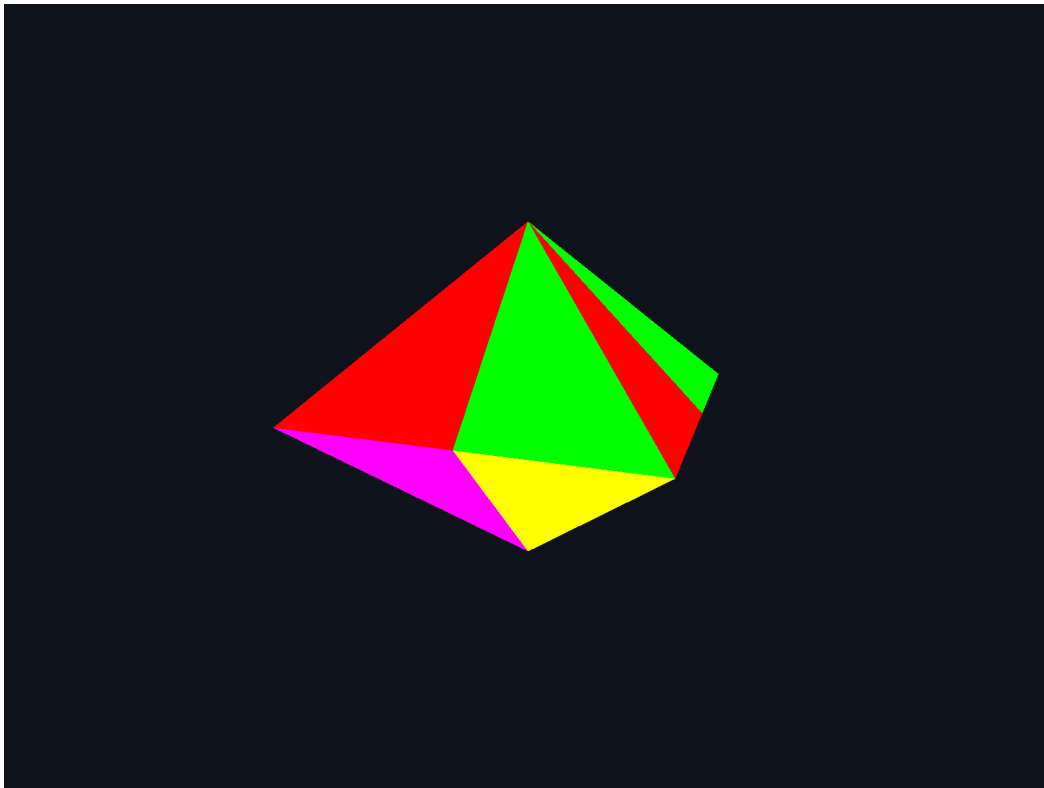


Figura 4. Rombo cerrado formado por las ocho pirámides.

Las bases azules quedan dentro de la unión. Para comprobarlas mantuve la vista individual, donde se puede girar una pirámide y revisar sus cinco caras.



Figura 5. Revisión de una pirámide por separado y de su base azul.

1.4 Modificación para cerrar el rombo

Mover ocho pirámides rectas dejaba varias puntas separadas. Para que las cuatro de cada mitad llegaran a un mismo punto, añadí una combinación de giros y escala que inclina cada pirámide. La base conserva su forma cuadrada; la punta queda sobre una de sus esquinas. Por eso las instancias resultantes son pirámides oblicuas.

Esta combinación no aparece literalmente en el código del profesor. La añadí para resolver la unión y la marqué como P12. Se implementó con `glm::translate`, `glm::rotate` y `glm::scale`, que ya se utilizaban en el ejemplo, sin crear clases o funciones de dibujo nuevas.

```
practica3.cpp
715 model = glm::translate(model, glm::vec3(-0.25000000f, 0.5f, 0.25000000f));
716 model = glm::rotate(model, glm::radians(-135.00000000f), glm::vec3(0.0f, 1.0f, 0.0f));
717 // Inclinar la punta sin alterar los vertices originales de la malla.
718 model = glm::rotate(model, glm::radians(-35.26438968f), glm::vec3(0.0f, 0.0f, 1.0f));
719 model = glm::scale(model, glm::vec3(1.41421356f, 0.70710678f, 1.0f));
720 model = glm::rotate(model, glm::radians(54.73561032f), glm::vec3(0.0f, 0.0f, 1.0f));
721 model = glm::rotate(model, glm::radians(135.00000000f), glm::vec3(0.0f, 1.0f, 0.0f));
722 // Orientacion de los colores de esta instancia.
723 model = glm::rotate(model, glm::radians(0.00000000f), glm::vec3(0.0f, 1.0f, 0.0f));
724 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
725 piramideColor.RenderMeshColor();
```

Figura 6. Transformaciones de una de las cuatro pirámides superiores.

Los giros de -35.26438968° y 54.73561032° , junto con los factores de escala 1.41421356 y 0.70710678, producen una inclinación lateral, también llamada cizallamiento. Los valores corresponden a la raíz de 2, su inverso y el ángulo cuya tangente es 1 dividido entre la raíz de 2. Los giros alrededor de Y orientan esa inclinación hacia el centro de cada cuadrante.

La traslación (± 0.25 , 0.5, ± 0.25) coloca las cuatro piezas superiores. Las otras cuatro repiten el mismo procedimiento y giran 180° alrededor de Z para formar la mitad inferior. En GLM, estas operaciones se multiplican sobre la matriz de entrada; cambiar su orden también cambia el resultado (GLM, s. f.).

Comprobé las coordenadas resultantes: las bases son cuadrados de lado 1 en $Y = 0$ y cada grupo comparte su punta. No hay ocho copias superpuestas del mismo volumen. Las cuatro piezas ocupan los cuatro cuadrantes de cada mitad. El arreglo original de posiciones de la pirámide no se modificó.

2 Problemas encontrados y solución

Unión de las pirámides. Corregí los huecos y las puntas separadas mediante la inclinación del apartado anterior, documentada como P12 en CAMBIOS.md.

Cantidad de vértices. MeshColor usaba la cantidad de componentes como número de vértices. Corregí vertexCount para dividir entre seis: tres componentes XYZ y tres RGB. Así, los 108 componentes de la pirámide corresponden a 18 vértices. glDrawArrays cuenta vértices, no valores float (Khronos Group, s. f.).

Cilindro y cono. Corregí los índices del cilindro, que recorrían componentes en lugar de vértices. En el cono quité un punto de cierre adicional y agregué la base. Ambos usan RenderMesh, GL_TRIANGLES, 48 sectores y radio 1; su tamaño final se ajusta con la escala.

Inicialización y shaders. Inicialicé los atributos pendientes de Window, añadí el retorno de Initialise y lo comprobé en el main. Comenté la validación del programa antes de enlazar un VAO, conservando las comprobaciones de compilación y enlace. Estos ajustes, junto con el conteo de vértices, están marcados como P04, P09, P10 y P11.

Otros ajustes. Añadí las teclas 1, 2 y 3, cambié el fondo y el título, y guardé las fuentes en UTF-8. Los shaders conservan su contenido original. La solución reúne los archivos en un proyecto de Visual Studio para Windows.

3 Conclusión

El cohete quedó construido con las cinco figuras solicitadas y el rombo con ocho pirámides de base cuadrada. Cerrar el rombo tomó más trabajo: repartir las piezas en dos niveles dejaba sus puntas separadas. La inclinación permitió unir las sin volver a definir la malla.

Revisar una figura por separado me ayudó a comprobar la base azul, que queda oculta al cerrar el rombo. También fue importante documentar la transformación añadida: aunque utilicé las mismas funciones del ejemplo, la combinación necesaria para inclinar las piezas no estaba resuelta en él.

4 Bibliografía

GLM. (s. f.). GLM_EXT_matrix_transform. <https://glm.g-truc.net/0.9.9/api/a00668.html>

Khronos Group. (s. f.). GLAPI/glDrawArrays. OpenGL Wiki.
<https://wikis.khronos.org/opengl/GLAPI/glDrawArrays>

Práctica 3: Modelado geométrico y cámara sintética. (2026). [Código C++ y shaders proporcionados en clase]. Facultad de Ingeniería, UNAM.